DAA LAB 8

Kshitij Kejriwal-252010005

1st April 2026

Quick Sort

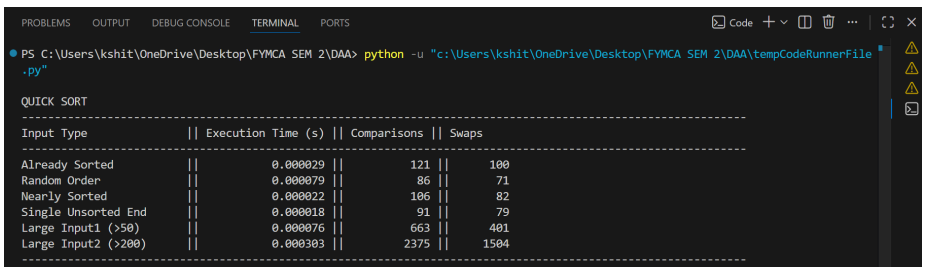
1 Code:

```python
import time
import random
import copy

class QuickSortAnalyzer:
    def __init__(self):
        self.comparisons = 0
        self.swaps = 0

    def reset(self):
        self.comparisons = 0
        self.swaps = 0

    def partition(self, arr, low, high):
        pivot_index = random.randint(low, high)
        arr[pivot_index], arr[high] = arr[high], arr[
            pivot_index]
        self.swaps += 1

        pivot = arr[high]
        i = low - 1

        for j in range(low, high):
            self.comparisons += 1
            if arr[j] < pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
                self.swaps += 1

        arr[i+1], arr[high] = arr[high], arr[i+1]
        self.swaps += 1
        return i + 1
```

```python

    def quick_sort(self, arr, low, high):
        if low < high:
            p = self.partition(arr, low, high)
            self.quick_sort(arr, low, p - 1)
            self.quick_sort(arr, p + 1, high)

    def sort_and_measure(self, arr):
        self.reset()
        start = time.perf_counter()
        self.quick_sort(arr, 0, len(arr) - 1)
        end = time.perf_counter()
        return end - start, self.comparisons, self.swaps


# -------- TEST CASES --------
test_cases = {
    "Already Sorted": list(range(1, 26)),
    "Reverse Sorted": list(range(25, 0, -1)),
    "Random Order": [16,1,4,2,12,9,10,3,5,24,14,20,6,23,7,25,19,18,8,22,11,17,13,15,21],
    "Nearly Sorted": [24,25,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,1,2],
    "Single Unsorted End": [2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,1],
    "Large Input1 (>50)": random.sample(range(1, 301), 80),
    "Large Input2 (>200)": random.sample(range(1, 1001), 300)
}

qs = QuickSortAnalyzer()
results = []

for name, data in test_cases.items():
    arr = copy.deepcopy(data)
    t, c, s = qs.sort_and_measure(arr)
    results.append((name, t, c, s))


print("\nQUICK SORT")
print("-" * 110)
print("Input Type                    || Execution Time (s) || Comparisons || Swaps")
print("-" * 110)

for r in results:
```

```python
    print(f"{r[0]:24} || {r[1]:18.6f} || {r[2]:12} || {r[3]:8}")

print("-" * 110)
```

2 Output:



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\kshit\OneDrive\Desktop\FYMCA SEM 2\DAA> python -u "c:\Users\kshit\OneDrive\Desktop\FYMCA SEM 2\DAA\tempCodeRunnerFile.py"

QUICK SORT
--------------------------------------------------------------------------------------------------------------
Input Type               || Execution Time (s) || Comparisons || Swaps
--------------------------------------------------------------------------------------------------------------
Already Sorted           ||          0.000029 ||         121 ||      100
Random Order             ||          0.000079 ||          86 ||       71
Nearly Sorted            ||          0.000022 ||         106 ||       82
Single Unsorted End      ||          0.000018 ||          91 ||       79
Large Input1 (>50)       ||          0.000076 ||         663 ||      401
Large Input2 (>200)      ||          0.000303 ||        2375 ||     1504
--------------------------------------------------------------------------------------------------------------
```

Figure 1: Output

3 Pseudocode:

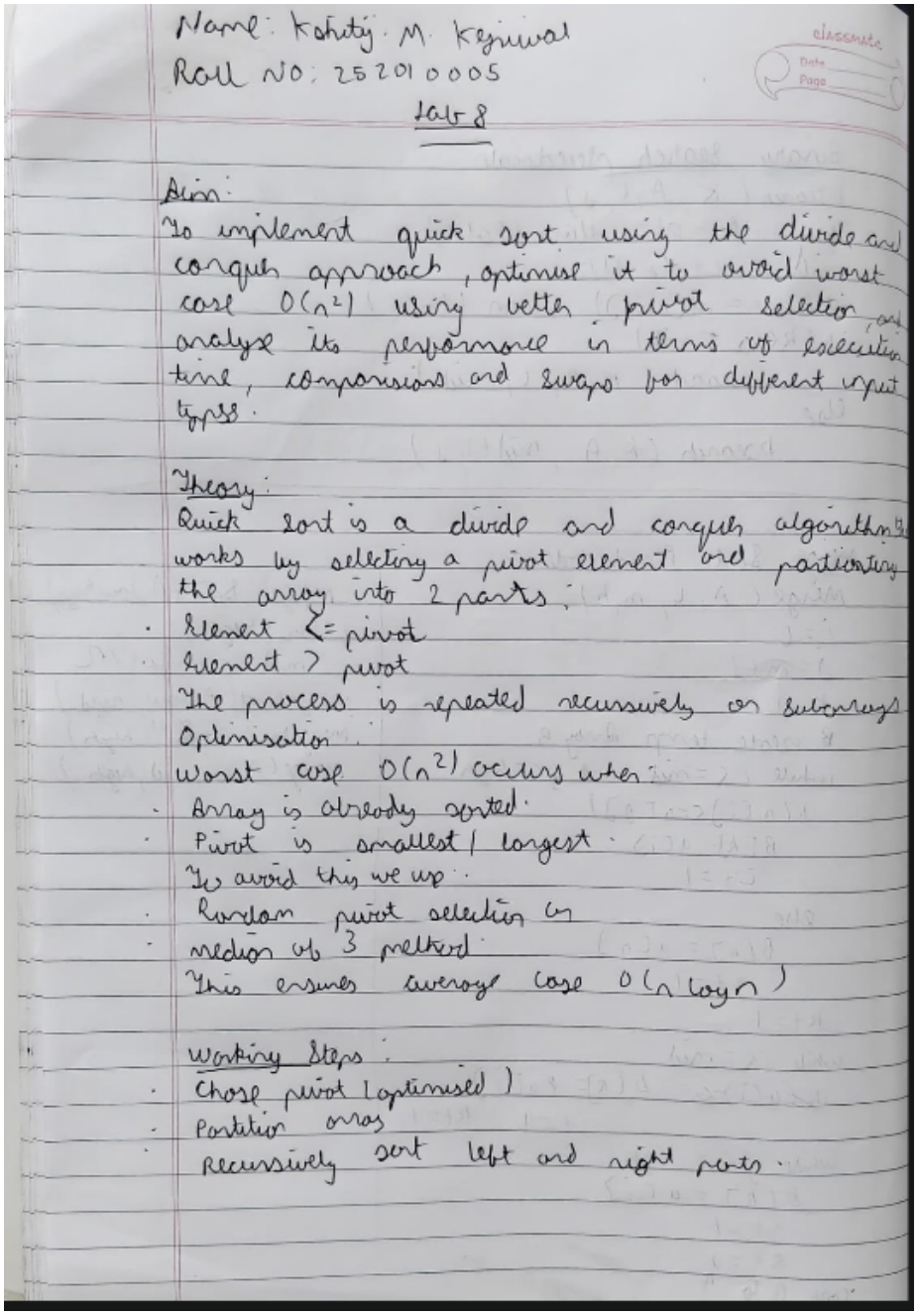
Algorithm 1 Optimized Quick Sort

```
procedure QuickSort(A, low, high)
    if low < high then
        p ← Partition(A, low, high)
        QuickSort(A, low, p − 1)
        QuickSort(A, p + 1, high)
    end if
end procedure
```

Algorithm 2 Partition with Random Pivot

```
procedure PARTITION(A, low, high)
    Choose random pivot index
    Swap pivot with A[high]
    pivot ← A[high]
    i ← low − 1
    for j ← low to high − 1 do
        if A[j] < pivot then
            i ← i + 1
            Swap A[i] and A[j]
        end if
    end for
    Swap A[i + 1] and A[high]
    return i + 1
end procedure
```

4 Explanation:



Name: Kshitij. M. Kejriwal
Roll NO: 2520100005

Lab 8

Aim:
To implement quick sort using the divide and conquer approach, optimise it to avoid worst case $O(n^2)$ using better pivot selection, and analyse its performance in terms of execution time, comparisons and swaps for different input types.

Theory:
Quick sort is a divide and conquer algorithm that works by selecting a pivot element and partitioning the array into 2 parts:
- Element <= pivot
- Element > pivot

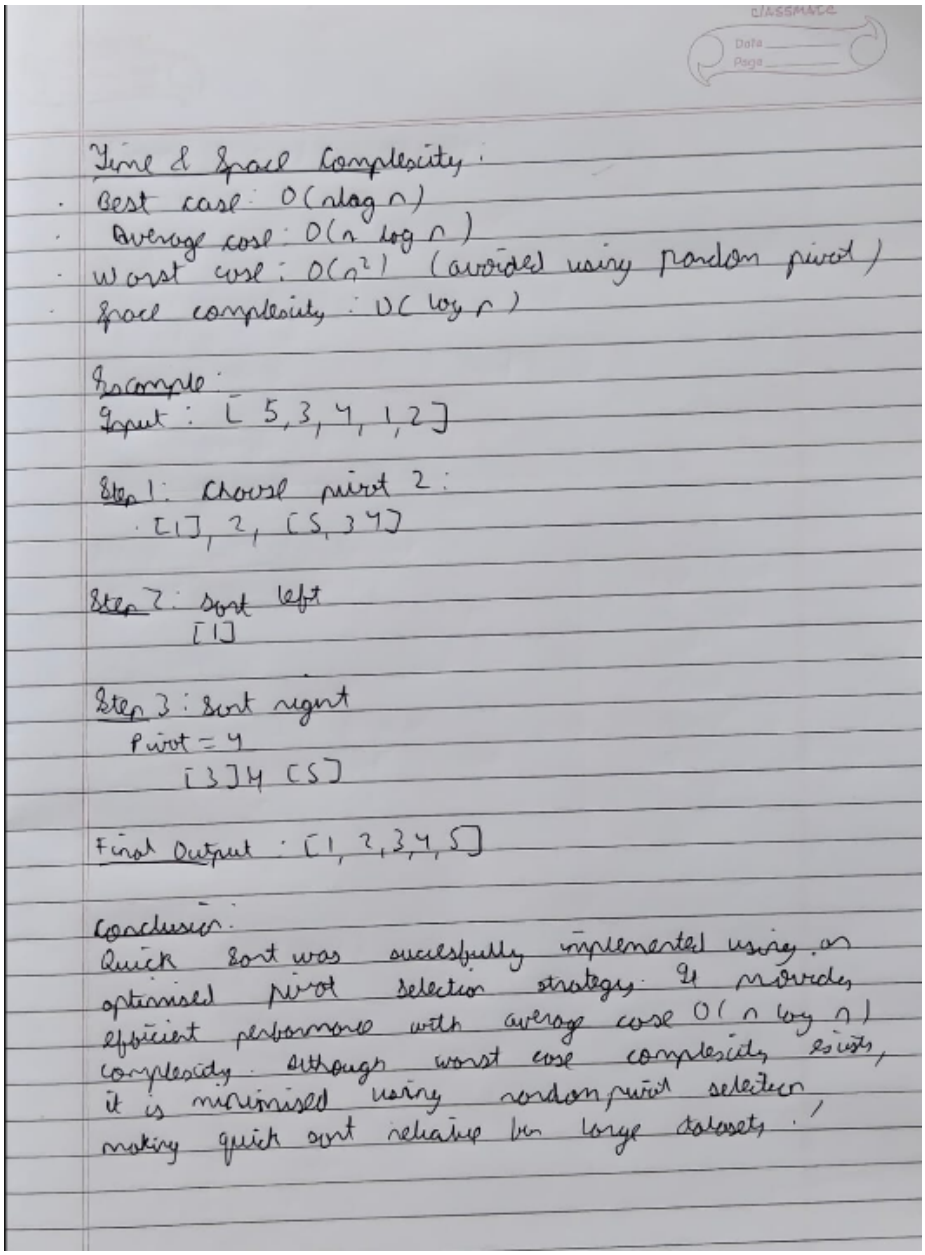
The process is repeated recursively on subarrays.
Optimisation:
Worst case $O(n^2)$ occurs when:
- Array is already sorted.
- Pivot is smallest / largest.

To avoid this we use:
- Random pivot selection or
- median of 3 method.

This ensures average case $O(n \log n)$

Working Steps:
- Chose pivot (optimised)
- Partition array
- Recursively sort left and right parts.

Figure 2: Explanation



Time & Space Complexity:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n^2)$ (avoided using random pivot)
- Space complexity: $O(\log n)$

Example:

Input: [5, 3, 4, 1, 2]

Step 1: Choose pivot 2:

[1], 2, [5, 3 4]

Step 2: Sort left

[1]

Step 3: Sort right

Pivot = 4

[3] 4 [5]

Final Output: [1, 2, 3, 4, 5]

Conclusion:

Quick Sort was successfully implemented using an optimised pivot selection strategy. It provides efficient performance with average case $O(n \log n)$ complexity. Although worst case complexity exists, it is minimised using random pivot selection, making quick sort reliable for large datasets.

Figure 3: Explanation